

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 4

1 Compiler Construction (jdy22)

lexical analysis
and syntax
analysis

- (a) Describe the inputs and outputs of a lexer and parser. [2 marks]

Answer: A lexer takes a stream of bytes (source code) as input and produces a stream of tokens. A parser takes a stream of tokens produced by a lexer and produces a syntax tree.

grammar
ambiguity, LL(k)

- (b) Here are three grammars for languages with branching, sequencing and variables:

Grammar 1: $E \rightarrow \text{if } E \text{ then } E \text{ else } E$
 $E \rightarrow E \text{ then } E \text{ end}$
 $E \rightarrow \text{id}$

Grammar 2: $E \rightarrow \text{if } E \text{ then } E \text{ else } E \text{ end}$
 $E \rightarrow E \text{ then } E \text{ end}$
 $E \rightarrow \text{id}$

Grammar 3: $E \rightarrow \text{if } E \text{ then } E \text{ else } E$
 $E \rightarrow \text{do } E \text{ then } E$
 $E \rightarrow \text{id}$

- (i) For each grammar, state whether it is ambiguous, giving an example if it is. [3 marks]

- (ii) For each grammar, state whether it is in LL(1), giving a reason if it is not. [3 marks]

Answer: (i) Grammar 1 is ambiguous, because `if w then x else y then z end` has two parses:

`(if w then x else y) then z end`
`if w then x else (y then z end)`

Grammars 2 and 3 are not ambiguous.

- (ii) Grammar 1 is not LL(1), because it's ambiguous and left-recursive. Grammar 2 is not LL(1), because it's left-recursive. Grammar 3 is LL(1).
-

bottom-up
parsing

- (c) Explain the roles of the ACTION and GOTO tables in the LR parsing algorithm, describing the entries and indexes for each table. [4 marks]

Answer: The ACTION table is indexed by the state at the top of the stack and the current input symbol. It indicates whether the next step should be to **shift** to a state **s**, **reduce** by a reduction $A \rightarrow \beta$, or **accept** the input (or fail).

The GOTO table is indexed by the state s at the top of the stack and a nonterminal A . It is used to determine the next state after a reduction $A \rightarrow \beta$ in state s .

- (d) Compilers sometimes simplify expressions to make type checking easier or to generate more efficient code. Here are two potential Slang simplifications:

expression	simplifies to	expression
<code>if true then e₁ else e₂</code>	$\rightarrow$	<code>e₁</code>
<code>(fun (x:t) -> e₁) e₂</code>	$\rightarrow$	<code>{e₂/x}e₁</code> (substitution)

A simplification $e_1 \rightarrow e_2$ is *correct for type checking* if e_1 and e_2 have the same type (or are both ill-typed), and *correct for optimization* if e_1 and e_2 have equivalent behaviour.

type checking

- (i) For each simplification, explain under what circumstances it is correct for type checking. [4 marks]

Answer: The `if` simplification is not correct for type checking in general. In the following cases the simplified expression has type `int`, but the original expression is ill-typed:

<code>if true then 2 else false</code>	$\rightarrow$	<code>2</code>
<code>if true then 2 else (1+false)</code>	$\rightarrow$	<code>2</code>

If e_1 and e_2 have the same type, or if e_1 is ill-typed then the `if` simplification is correct.

The `fun` simplification is not correct for type checking in general. In the following cases the simplified expression has type `int` but the original expression is ill-typed:

<code>(fun (x:int) -> 2) (1+true)</code>	$\rightarrow$	<code>2</code>
<code>(fun (x:bool) -> x+2) 1</code>	$\rightarrow$	<code>1+2</code>
<code>(fun (x:int) -> 2) true</code>	$\rightarrow$	<code>2</code>

However, the `fun` simplification is correct for type checking under some circumstances, e.g. if e_2 has type `t` and e_1 is well-typed in an environment extended with $x:t$, or if x is free in e_1 and e_2 is ill-typed.

optimization

- (ii) For each simplification, explain under what circumstances it is correct for optimization. [4 marks]

Answer: The `if` simplification is correct for optimization.

The `fun` simplification is correct for optimization if e_2 does not have side effects.

For example, simplifying `(fun (x:int) -> x + ?) 3` to `3 + ?` is correct for optimization purposes, because the simplification preserves the evaluation order.

However, simplifying `(fun (x:int) -> ? - x) ?` to `? - ?` is not correct for optimization purposes, because the optimization changes the evaluation order to read the minuend before the subtrahend.